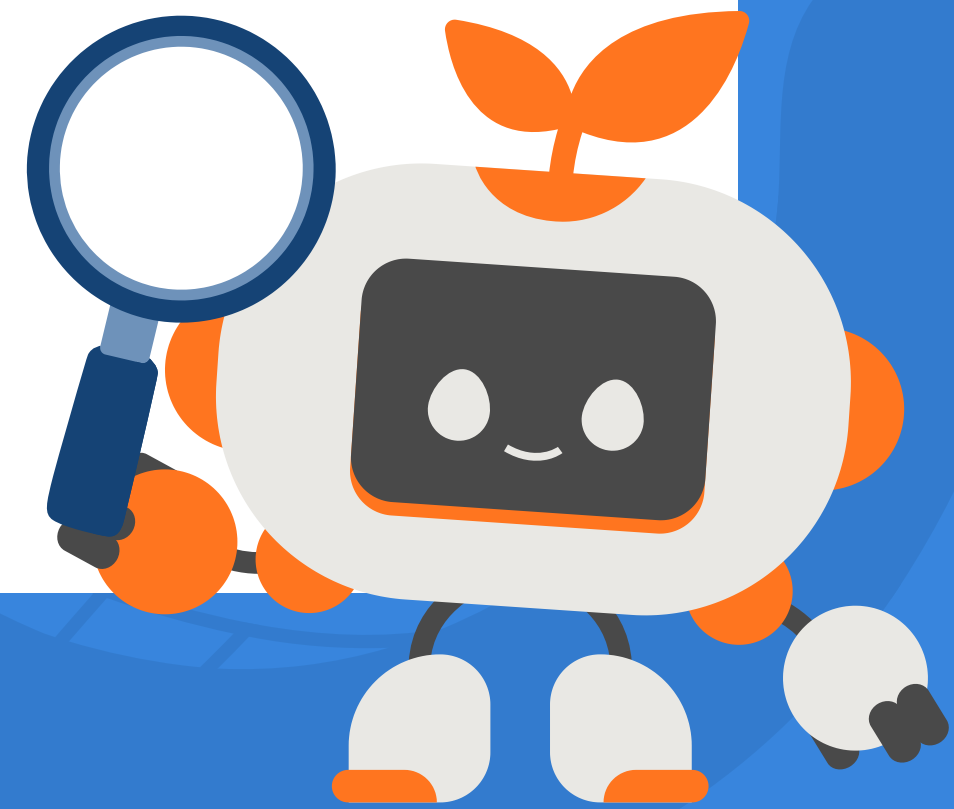


PERANCANGAN DAN IMPLEMENTASI BASIS DATA SISTEM PENGGAJIAN KARYAWAN MENGGUNAKAN MY SQL

kelompok E Basis Data



ANGGOTA KELOMPOK

KELOMPOK E BASIS DATA



**Kinanti
Jingga
Adzhani**

K1D024007



**Natasya
Jamila
Mapaza**

K1D024016



**Muflihah
Qurrota Ain
Az'zahra**

K1D024024



**Muhammad
Alief
Badrudin**

K1D024030



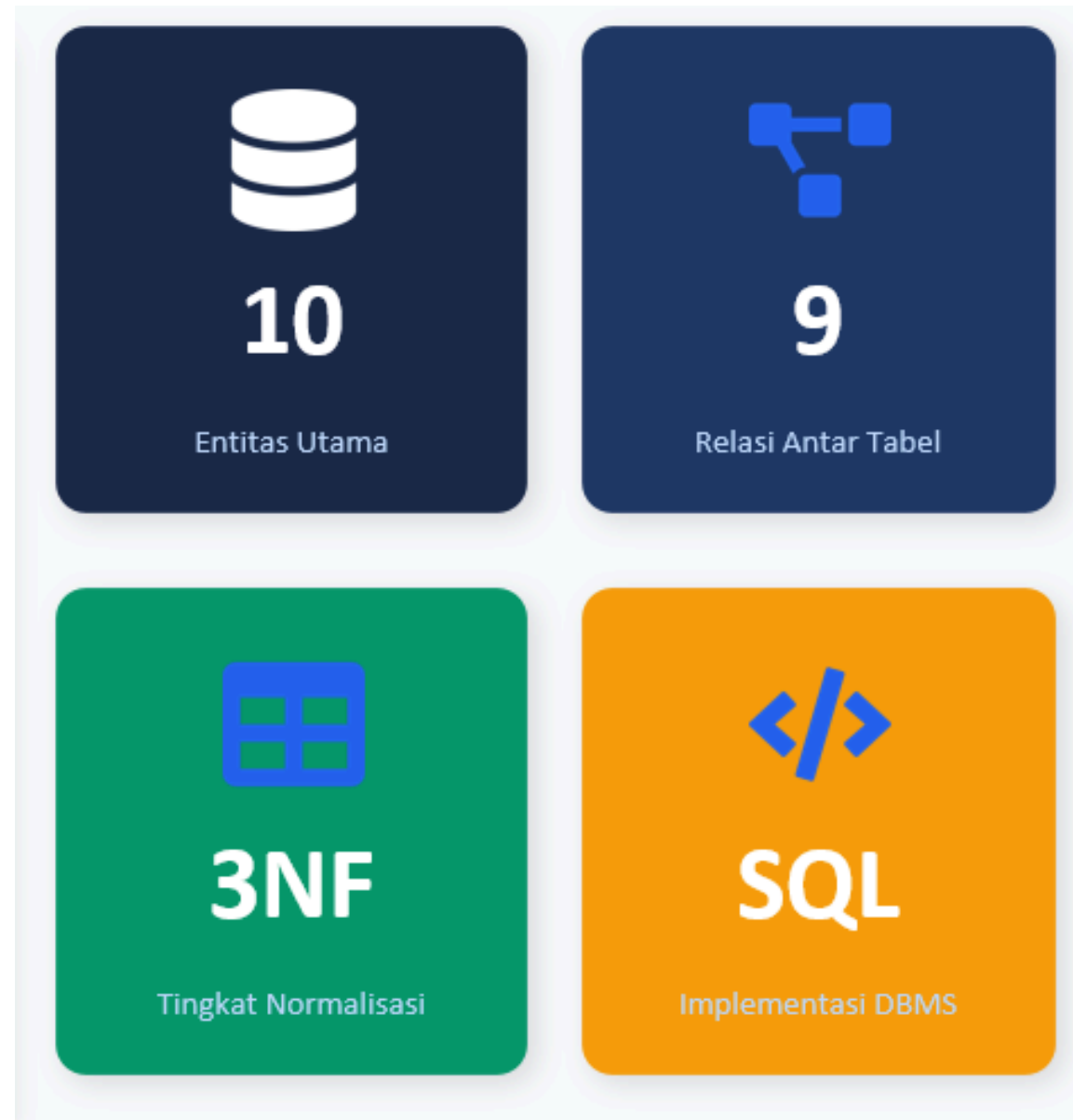
**Hafdsa
Syafiya**

K1D024040



LATAR BELAKANG

- Pengelolaan data penggajian karyawan merupakan proses penting dalam operasional perusahaan karena berkaitan dengan hak dan kesejahteraan karyawan.
- Pengolahan data secara manual berisiko menimbulkan kesalahan perhitungan, duplikasi data, serta keterlambatan dalam penyajian informasi.
- Basis data yang terintegrasi dapat meningkatkan akurasi, konsistensi, dan efisiensi dalam pengelolaan data penggajian.
- Oleh karena itu, dirancang menggunakan MySQL untuk mendukung proses secara efektif.



Tujuan Project

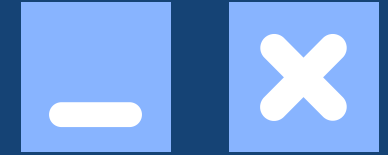


- Merancang basis data yang sesuai untuk mendukung Sistem Penggajian Karyawan.
- Mengidentifikasi entitas, atribut, dan relasi yang terdapat dalam Sistem Penggajian Karyawan.
- Menerapkan proses normalisasi untuk menghasilkan struktur basis data yang efisien.
- Mengimplementasikan basis data Sistem Penggajian Karyawan menggunakan MySQL.

• • • Ruang Lingkup Sistem

- Perancangan basis data Sistem Penggajian Karyawan.
- Pembuatan ERD (Entity Relationship Diagram).
- Proses normalisasi hingga bentuk 3NF.
- Implementasi tabel dan relasi pada MySQL.
- Pengelolaan data dan Implementasi query SQL, function, view, dan trigger.

Analisis Kebutuhan



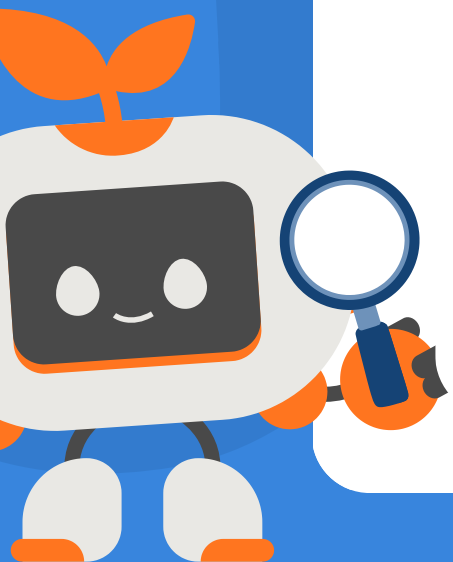
Sistem penggajian dibutuhkan untuk mengelola data karyawan, absensi, lembur, tunjangan, potongan, dan penggajian secara terintegrasi sehingga proses perhitungan gaji menjadi lebih cepat, akurat, dan terdokumentasi dengan baik.

Kebutuhan Fungsional

- Mengelola data departemen, jabatan, dan karyawan.
- Mengelola data kontrak dan rekening bank.
- Mencatat absensi dan lembur.
- Mengelola tunjangan dan potongan.
- Menghitung serta menyimpan data penggajian.
- Menampilkan laporan penggajian.

Kebutuhan NonFungsional

- Keamanan data terjamin.
- Data konsisten dan terintegrasi.
- Perhitungan gaji akurat.
- Sistem mudah digunakan.
- Pengolahan data cepat dan efisien.
- Mudah dikembangkan dan dipelihara.



ERD

Entitas Yang Digunakan

Departemen

Menyimpan data departemen perusahaan

Jabatan

Menyimpan data jabatan dan rentang gaji

Karyawan

Menyimpan identitas karyawan

Kontrak

Menyimpan informasi kontrak kerja karyawan

Rekening Bank

Menyimpan data rekening penerima gaji karyawan

Absen

Menyimpan data kehadiran karyawan

Lembur

Menyimpan data lembur karyawan

Penggajian

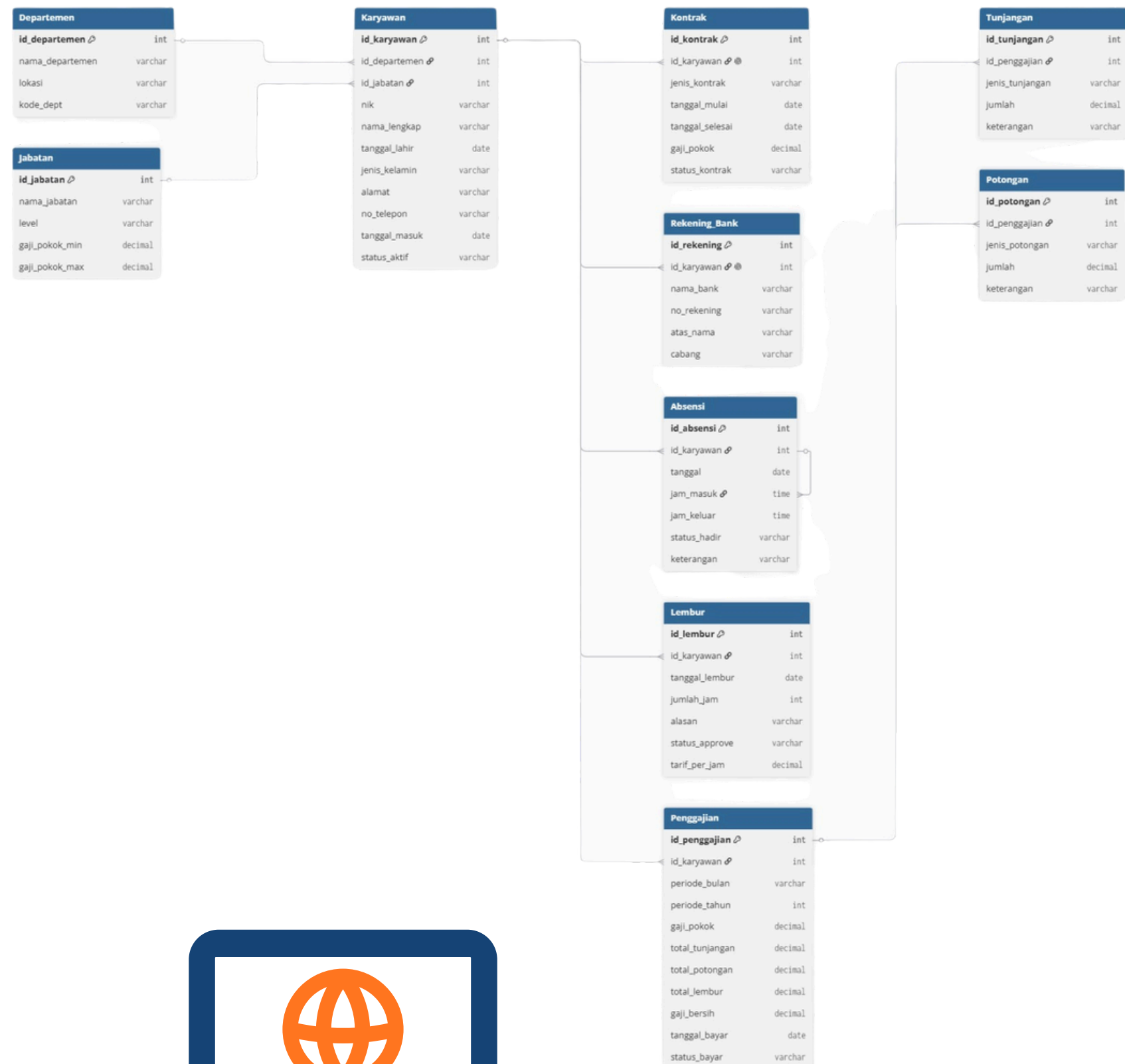
Menyimpan transaksi penggajian

Tunjangan

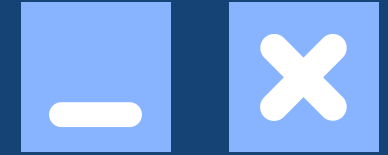
Menyimpan komponen tunjangan

Potongan

Menyimpan komponen potongan

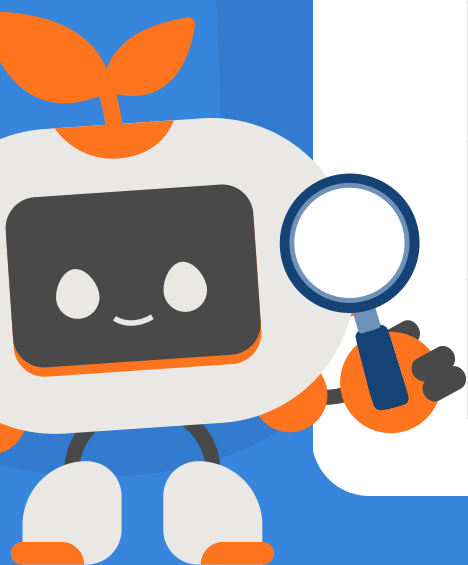


Relasi Antar Tabel



Relasi antar entitas digunakan untuk menunjukkan hubungan yang terjadi antara satu entitas dengan entitas lainnya dalam Sistem Penggajian Karyawan. Relasi ini menjelaskan bagaimana data pada suatu entitas saling berhubungan dan mendukung proses penggajian dalam perusahaan.

No	Entitas 1	Relasi	Entitas 2	Kardinalitas
1	DEPARTEMEN	Memiliki	KARYAWAN	1 : N
2	JABATAN	Memiliki	KARYAWAN	1 : N
3	KARYAWAN	Memiliki	KONTRAK	1 : 1
4	KARYAWAN	Memiliki	REKENING_BANK	1 : 1
5	KARYAWAN	Memiliki	ABSENSI	1 : N
6	KARYAWAN	Melakukan	LEMBUR	1 : N
7	KARYAWAN	Menerima	PENGGAJIAN	1 : N
8	PENGGAJIAN	Mencakup	TUNJANGAN	1 : N
9	PENGGAJIAN	Mencakup	POTONGAN	1 : N





Normalisasi (1NF-3NF)



1NF

Syarat:

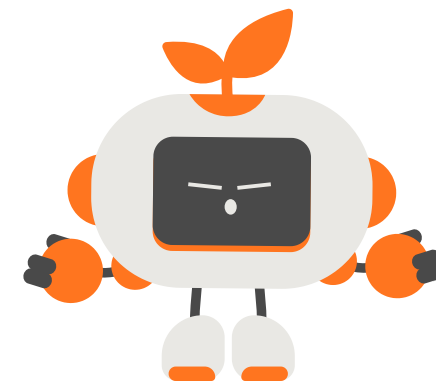
Setiap kolom hanya boleh berisi satu nilai
(tidak boleh lebih dari satu data dalam satu kolom)

✗ Sebelum

id_karyawan	nama	jabatan
K110	Andi	staff,admin

✓ Sesudah

id_jabatan	nama jabatan
J01	Staff
J02	Admin



2NF

Syarat:

Data yang tidak berhubungan langsung dengan primary key harus dipindahkan ke tabel yang sesuai

✗ Sebelum

Id_gaji	id_karyawan	nama_karyawan	gaji
G001	K110	Andi	5.000.000

✓ Sesudah

id_karyawan	nama_karyawan
K110	Andi

id_gaji	id_karyawan	gaji
G001	K110	5.000.000



Normalisasi (1NF-3NF)



3NF

Syarat:

Sudah dalam 2NF, dan tidak ada transitive dependency (atribut non-key tidak bergantung pada atribut non-key lain).

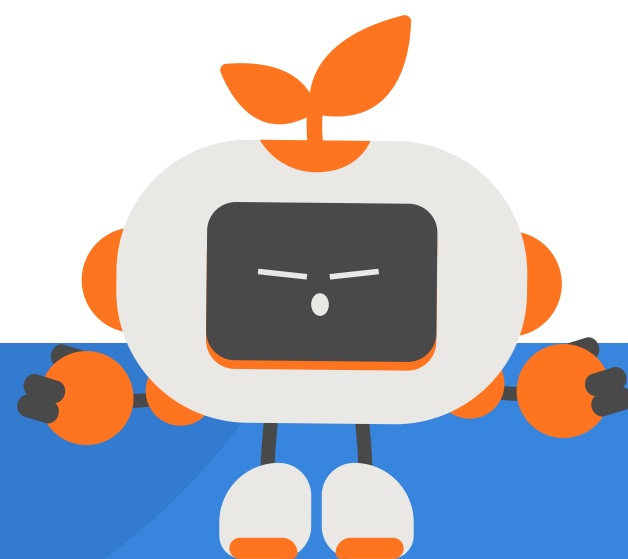
✗ Sebelum

KARYAWAN menyimpan nama_departemen (bergantung pada id_departemen, bukan id_karyawan)

✓ Sesudah

Pisahkan ke tabel DEPARTEMEN.

KARYAWAN hanya simpan id_departemen (FK)



✗ Sebelum (melanggar 3NF)

KARYAWAN

id_karyawan	nama_karyawan	id_departemen	nama_departemen
K001	Sari	D01	Keuangan
K002	Budi	D01	Keuangan

nama_departemen bergantung pada id_departemen, bukan langsung pada id_karyawan → transitive dependency.

✓ Sesudah (memenuhi 3NF)

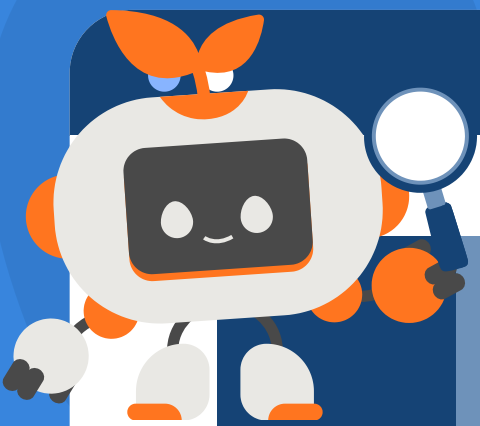
KARYAWAN

id_karyawan	nama_karyawan	id_departemen (FK)
K001	Sari	D01
K002	Budi	D01

id_departemen menjadi foreign key di KARYAWAN, menghubungkan ke tabel DEPARTEMEN → tidak ada lagi transitive dependency.

DEPARTEMEN

id_departemen (PK)	nama_departemen
D01	Keuangan
D02	Pemasaran



Implementasi DDL



Implementasi DDL dilakukan untuk membangun struktur database Sistem Penggajian Karyawan menggunakan MySQL. Pada tahap ini dibuat 10 tabel utama, yaitu Departemen, Jabatan, Karyawan, Kontrak, Rekening_Bank, Absensi, Lembur, Penggajian, Tunjangan, dan Potongan.

Setiap tabel dirancang berdasarkan hasil normalisasi hingga bentuk normal ketiga (3NF) dan dihubungkan menggunakan Primary Key (PK) serta Foreign Key (FK) untuk menjaga integritas data.

Selain itu, diterapkan beberapa constraint seperti NOT NULL, UNIQUE, dan CHECK untuk memastikan data yang disimpan valid dan tidak terjadi duplikasi data.

```
CREATE TABLE Karyawan (  
  id_karyawan VARCHAR(10) PRIMARY KEY,  
  nik VARCHAR(20) UNIQUE,  
  nama_lengkap VARCHAR(100) NOT NULL,  
  id_departemen VARCHAR(10),  
  id_jabatan VARCHAR(10),  
  FOREIGN KEY (id_departemen)  
    REFERENCES Departemen(id_departemen),  
  FOREIGN KEY (id_jabatan)  
    REFERENCES Jabatan(id_jabatan)  
);
```

Table: **karyawan**

Columns:

id_karyawan	varchar(10) PK
nik	varchar(20)
nama_lengkap	varchar(100)
tanggal_lahir	date
jenis_kelamin	enum('L','P')
alamat	varchar(255)
no_telepon	varchar(20)
tanggal_masuk	date
status_aktif	enum('Aktif','Nonaktif')
id_departemen	varchar(10)
id_jabatan	varchar(10)

Komponen	Keterangan
id_karyawan	Primary Key yang digunakan sebagai identitas unik setiap karyawan
nik	Data NIK karyawan dan tidak boleh duplikat (UNIQUE)
nama_lengkap	Nama karyawan dan wajib diisi (NOT NULL)
id_departemen	Foreign Key yang menghubungkan tabel Karyawan dengan Departemen
id_jabatan	Foreign Key yang menghubungkan tabel Karyawan dengan Jabatan

Implementasi DML dan Query Kompleks

Contoh: Data Karyawan

-- 3. KARYAWAN

INSERT INTO Karyawan

(id_karyawan, nik, nama_lengkap, tanggal_lahir, jenis_kelamin, alamat, no_telepon, tanggal_masuk, status_aktif, id_departemen, id_jabatan)

VALUES

('K001','3301010101800001','Ahmad Fauzi','1980-01-01','L','Jl. Merdeka No.1, Purwokerto','081200001111','2018-03-01','Aktif','D001','J002'),
('K002','3301010201850002','Siti Rahayu','1985-02-02','P','Jl. Sudirman No.5, Purwokerto','081200002222','2019-05-15','Aktif','D002','J004'),

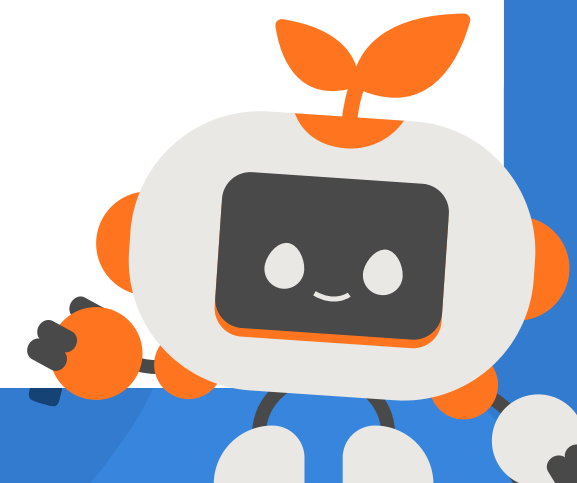
id_karyawan	nik	nama_lengkap	tanggal_lahir	jenis_kelamin	alamat	no_telepon	tanggal_masuk	status_aktif	id_departemen	id_jabatan
K001	3301010101800000	Ahmad Fauzi	1/1/1980	L	Jl. Merdeka No.1, Purwokerto	81200001111	1/3/2018	Aktif	D001	J002
K002	3301010201850000	Siti Rahayu	2/2/1985	P	Jl. Sudirman No.5, Purwokerto	81200002222	15/5/2019	Aktif	D002	J004

Ringkasan Data yang Diinputkan:

Departemen	5
Jabatan	7
Karyawan	30
Rekening Bank	30

Kontrak	30
Absensi	34
Lembur	15
Penggajian	30

Tunjangan	34
Potongan	30



Query Select, Where & Join

--Lembur yang Sudah Disetujui dengan Jam Lembur Minimal 3 Jam

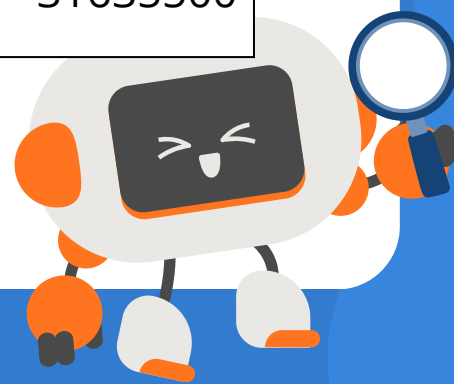
```
SELECT
  id_lembur,
  id_karyawan,
  tanggal_lembur,
  jumlah_jam,
  tarif_per_jam
FROM Lembur
WHERE status_approve = 'Approved'
AND jumlah_jam >= 3
ORDER BY jumlah_jam DESC;
```

id_lembur	id_karyawan	tanggal_lembur	jumlah_jam	tarif_per_jam
LBR006	K009	13/4/2025	5	27500
LBR003	K004	10/4/2025	4	22000
LBR009	K019	18/4/2025	4	27500
LBR001	K001	8/4/2025	3	25000

--- Menampilkan informasi pengeluaran gaji pada setiap departemen

```
SELECT
  d.nama_departemen,
  COUNT(k.id_karyawan) AS jumlah_karyawan,
  ROUND(AVG(p.gaji_bersih),0) AS rata_rata_gaji,
  SUM(p.gaji_bersih) AS total_pengeluaran_gaji
FROM Departemen d
INNER JOIN Karyawan k
  ON d.id_departemen = k.id_departemen
INNER JOIN Penggajian p
  ON k.id_karyawan = p.id_karyawan
GROUP BY d.nama_departemen
HAVING SUM(p.gaji_bersih) > 30000000
ORDER BY total_pengeluaran_gaji DESC;
```

nama_departemen	jumlah_karyawan	rata_rata_gaji	total_pengeluaran_gaji
Sumber Daya Manusia	6	9921167	59527000
Keuangan	6	6438833	38633000
Pemasaran	5	6660400	33302000
Teknologi Informasi	7	4519357	31635500



-- 1. View Informasi Penggajian Lengkap Karyawan

CREATE VIEW vw_penggajian_lengkap AS

SELECT

p.id_penggajian,
k.id_karyawan,
k.nama_lengkap,
d.nama_departemen,
j.nama_jabatan,
p.gaji_bersih,
p.tanggal_bayar,
p.status_bayar

FROM Penggajian p

INNER JOIN Karyawan k

ON p.id_karyawan = k.id_karyawan

INNER JOIN Departemen d

ON k.id_departemen = d.id_departemen

INNER JOIN Jabatan j

ON k.id_jabatan = j.id_jabatan;

SELECT * FROM vw_penggajian_lengkap;

id_penggaj	id_karyaw	nama_lengk	nama_departe	nama_jabat	gaji_bersi	tanggal_bayar	status_bayar
PGJ001	K001	Ahmad	Sumber Daya	Manajer	11325000	30/4/2025	Lunas
PGJ006	K006	Fitria Sari	Sumber Daya	Supervisor	6229000	30/4/2025	Lunas
PGJ012	K012	Linda	Sumber Daya	Manajer	10725000	30/4/2025	Lunas

-- Menampilkan karyawan yang memiliki gaji bersih
-- lebih tinggi dibandingkan rata-rata gaji seluruh karyawan

SELECT

k.id_karyawan,
k.nama_lengkap,
p.gaji_bersih

FROM Karyawan k

INNER JOIN Penggajian p

ON k.id_karyawan = p.id_karyawan

WHERE p.gaji_bersih >

(

SELECT AVG(gaji_bersih)
FROM Penggajian

)

ORDER BY p.gaji_bersih DESC;

Query Lanjutan

```
DELIMITER $$
CREATE PROCEDURE HitungTotalTunjangan(
  IN p_id_penggajian VARCHAR(10)
)
BEGIN
  DECLARE v_total_tunjangan DECIMAL(12,2);
  -- Menghitung total tunjangan
  SELECT COALESCE(SUM(jumlah_tunjangan),0)
  INTO v_total_tunjangan
  FROM Tunjangan
  WHERE id_penggajian = p_id_penggajian;
  -- Mengupdate tabel Penggajian
  UPDATE Penggajian
  SET total_tunjangan = v_total_tunjangan
  WHERE id_penggajian = p_id_penggajian;
END$$
DELIMITER ;
-- 1. Lihat data tunjangan
SELECT *
FROM Tunjangan
WHERE id_penggajian = 'PGJ002';
-- 2. Jalankan Stored Procedure
CALL HitungTotalTunjangan('PGJ002');
-- 3. Lihat hasil update
SELECT
  id_penggajian,
  total_tunjangan
FROM Penggajian
WHERE id_penggajian = 'PGJ002';
```

id_penggajian	total_tunjangan
PGJ002	120.000.000

Stored Procedure (HitungTotalTunjangan)

Fungsi: Untuk menghitung total tunjangan seorang karyawan pada satu ID penggajian secara otomatis dari tabel Tunjangan, lalu langsung mengupdate hasilnya ke tabel Penggajian.

```
DELIMITER $$
CREATE TRIGGER trg_validasi_absensi
BEFORE INSERT ON Absensi
FOR EACH ROW
BEGIN
  IF NEW.status_hadir = 'Hadir'
    AND (NEW.jam_masuk IS NULL OR
    NEW.jam_keluar IS NULL)
  THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT =
    'Karyawan yang hadir wajib memiliki jam masuk
    dan jam keluar';
  END IF;
END$$
DELIMITER ;
-- Tampilkan TRIGGER
SHOW TRIGGERS;
-- Uji Data Salah
INSERT INTO Absensi
VALUES
(
  'ABS101',
  'K002',
  '2025-04-30',
  NULL,
  NULL,
  'Hadir',
  '-'
);
```

Query Lanjutan

```
-- Uji Data Benar
INSERT INTO Absensi
VALUES
(
  'ABS102',
  'K002',
  '2025-04-30',
  '08:00:00',
  '17:00:00',
  'Hadir',
  '-'
);
-- Cek Data
SELECT *
FROM Absensi
WHERE id_absensi IN ('ABS101','ABS102');
```

id_abse nsi	id_karya wan	tanggaal	jam_ma suk	jam_kel uar	status_h adir	keteran gan_abs
ABS102	K002	30/4...	08:00:00	17:00:00	Hadir	-
NULL	NULL	NULL	NULL	NULL	NULL	NULL

Trigger (trg_validasi_absensi)

Fungsi: Sebagai validasi otomatis. Jika ada admin yang menginput data absensi dengan status 'Hadir', tapi jam masuk atau jam keluarnya dikosongkan (NULL), database akan otomatis menolak (SIGNAL SQLSTATE '45000') dan memunculkan pesan error.

```
DELIMITER $$
CREATE FUNCTION HitungUsia(
  p_tanggal_lahir DATE
)
RETURNS INT
DETERMINISTIC
BEGIN
  DECLARE v_usia INT;
  SET v_usia = TIMESTAMPDIFF(
    YEAR,
    p_tanggal_lahir,
    CURDATE()
  );
  RETURN v_usia;
END$$
DELIMITER ;
```

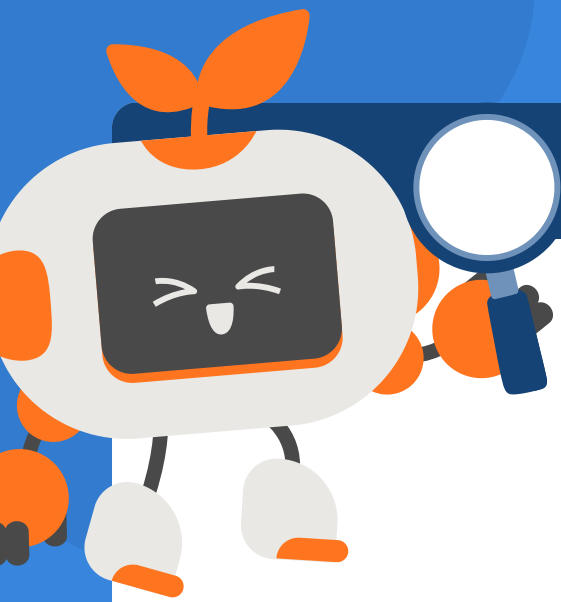
```
--
SELECT
  id_karyawan,
  nama_lengkap,
  tanggal_lahir,
  HitungUsia(tanggal_lahir) AS usia
FROM Karyawan;
```

```
SHOW FUNCTION STATUS
WHERE Db = 'sistem_penggajian';
SELECT
  nama_lengkap,
  tanggal_lahir,
  HitungUsia(tanggal_lahir) AS usia
FROM Karyawan
LIMIT 5;
SELECT
  id_karyawan,
  nama_lengkap,
  tanggal_lahir,
  HitungUsia(tanggal_lahir) AS usia
FROM Karyawan
ORDER BY usia DESC
LIMIT 5;
```

id_karyawan	nama_lengkap	tanggal_lahir	usia
K001	Ahmad Fauzi	1/1/1980	46
K025	Zainal Arifin	25/1/1980	46
K016	Putri Handayani	16/4/1981	45
K027	Bagus Setiawan	27/3/1983	43
K008	Heni Purwanti	8/8/1983	42

User-Defined Function (HitungUsia)

Fungsi: Fungsi kustom buatan sendiri untuk menghitung usia karyawan saat ini secara otomatis berdasarkan rumus selisih tahun antara tanggal_lahir dengan tanggal hari ini (CURDATE()).



Kesimpulan

- Berhasil merancang dan mengimplementasikan basis data Sistem Penggajian Karyawan menggunakan MySQL.
- Basis data terdiri dari 10 entitas yang telah dinormalisasi hingga 3NF sehingga mengurangi redundansi data.
- Implementasi DDL, DML, Query SQL, Function, dan View berjalan dengan baik.
- Sistem mampu menyajikan informasi penggajian secara akurat, terintegrasi, dan efisien untuk mendukung kebutuhan pengelolaan data perusahaan.

Terima Kasih!

Sampai jumpa di kelas berikutnya.

